Documento de Inicio de Proyecto (Kick-off)

Proyecto: Sistema de Gestión Comercial para PyMEs (MVP)
Fecha: Octubre 2026

1. Visión General del Proyecto

Vamos a desarrollar un Sistema de Punto de Venta (POS) y Gestión de Stock enfocado en comercios minoristas. El objetivo de este MVP es resolver tres problemas críticos del comerciante: velocidad en el mostrador, control estricto de inventario y claridad financiera (caja).

No es solo un proyecto de código, es un producto pensado para comercializarse bajo un modelo de instalación inicial + abono mensual de mantenimiento y soporte (SaaS on-premise/cloud híbrido).

La aplicación será de escritorio, utilizando Electron para ejecutar el frontend y permitir la integración con el sistema operativo. El backend estará desarrollado en Java con Spring Boot y será responsable de la lógica de negocio, la seguridad y las operaciones sobre la base de datos.

2. Stack Tecnológico Acordado

Para garantizar velocidad de desarrollo, seguridad en transacciones y una interfaz fluida, utilizaremos las siguientes tecnologías:

- **Backend & Framework:** Java 21 LTS con Spring Boot, utilizando una arquitectura por capas (Controller, Service y Repository).
- **Gestión de dependencias:** Maven.
- **Persistencia:** Spring Data JPA con Hibernate para el mapeo objeto-relacional (ORM).
- **Base de Datos:** MySQL / MariaDB (motor InnoDB para garantizar transacciones ACID).
- **Migraciones de Base de Datos:** Flyway para versionar y ejecutar cambios en el esquema de forma controlada.
- **Frontend:** React con JavaScript (JSX), HTML5 y Bootstrap 5.
- **Aplicación de escritorio:** Electron para la ejecución de React como aplicación de escritorio, integración con el sistema operativo e impresión de tickets.
- **Herramienta de construcción del Frontend:** Vite.

- **Interactividad (POS):** JavaScript y Fetch API para realizar búsquedas asíncronas, actualizar el carrito y comunicarse con el backend sin recargar la interfaz.
- **API:** REST con respuestas y solicitudes en formato JSON.
- **Seguridad:** Spring Security para autenticación y autorización; contraseñas almacenadas mediante hashing seguro.
- **Validaciones:** Jakarta Bean Validation para validar los datos recibidos por el backend.
- **Documentación de API:** OpenAPI / Swagger.
- **Pruebas:** JUnit 5, Mockito y Spring Boot Test.
- **Control de Versiones:** Git y GitHub.

El frontend utilizará JavaScript, no TypeScript. Los componentes de React se desarrollarán en archivos .jsx y los servicios auxiliares en archivos .js.

3. Equipo y Distribución de Roles

El desarrollo será ejecutado por nosotros dos, con el respaldo de un Asesor Técnico Senior (IA) que nos guiará en la arquitectura y resolución de bloqueos complejos.

Desarrollador A (Backend y Base de Datos)

- Diseño de las entidades, relaciones y migraciones de la base de datos.
- Implementación de entidades JPA, repositorios y servicios mediante Spring Data JPA y Hibernate.
- Creación de controladores REST y DTO para la comunicación con el frontend.
- Validación de las reglas de negocio y los datos recibidos.
- Manejo de transacciones mediante @Transactional para evitar inconsistencias entre ventas, pagos y stock.
- Implementación de bloqueos de filas cuando sea necesario para prevenir ventas concurrentes de existencias insuficientes.
- Desarrollo de la autenticación, autorización y permisos mediante Spring Security.
- Implementación de las operaciones de apertura, movimientos y cierre de caja.

- Desarrollo de pruebas unitarias y de integración.
- Documentación de los endpoints mediante OpenAPI / Swagger.

Desarrollador B (Frontend y Experiencia de Usuario)

- Maquetación de las interfaces mediante React, JSX, HTML5, CSS y Bootstrap 5.
- Desarrollo de la aplicación de escritorio con Electron.
- Configuración del entorno de desarrollo y construcción mediante Vite.
- Desarrollo de la interfaz del Punto de Venta (POS), optimizada para uso con teclado, atajos y lector de código de barras.
- Implementación de las peticiones asíncronas mediante Fetch API para el buscador de productos, el carrito y la comunicación con la API REST.
- Desarrollo de las pantallas de productos, categorías, proveedores, inventario, caja y reportes.
- Implementación de mensajes de error, validaciones visuales y confirmaciones de operaciones.
- Integración de la impresión de tickets mediante Electron.
- Desarrollo de una interfaz responsiva y consistente.
- Uso de datos falsos (mock data) o datos de prueba para avanzar en paralelo con el backend.

Asesor Técnico Senior (Rol de Soporte IA)

- Proveerá los diseños de la base de datos y los scripts SQL necesarios.
- Validará la arquitectura general del sistema y la separación de responsabilidades.
- Revisará la seguridad de las transacciones y la integridad de los datos.
- Realizará revisiones de código (Code Reviews).
- Ayudará a resolver bloqueos técnicos y problemas de integración.
- Revisará consultas lentas y propondrá optimizaciones.
- Asistirá en el diseño de pruebas de concurrencia y casos límite.
- Ayudará a definir contratos de API, DTO y estándares de desarrollo.
- Asistirá en la preparación de la instalación y las pruebas del piloto comercial.

4. Metodología y Flujo de Trabajo

Para poder trabajar en paralelo sin pisarnos el código en GitHub, seguiremos esta regla de oro:

1. **Diseño Data-First:** Lo primero que definiremos y aprobaremos juntos será el esquema de la base de datos, incluyendo entidades, relaciones, claves foráneas, índices y restricciones de integridad.
2. **Configuración Inicial:** Crearemos el repositorio en GitHub y configuraremos el backend con Spring Boot y Maven, el frontend con React, JavaScript, Vite y Electron, y la conexión con MySQL/MariaDB.
3. **Migraciones y Modelos:** El Desarrollador A implementará las migraciones mediante Flyway, las entidades JPA, los repositorios y los datos iniciales de prueba.
4. **Trabajo Paralelo:** Una vez definidos el modelo de datos y los contratos de la API, el Desarrollador A construirá los endpoints y la lógica de negocio, mientras que el Desarrollador B desarrollará las interfaces con datos falsos (mock data) hasta disponer de los endpoints definitivos.
5. **Contratos de API:** Antes de integrar cada módulo, acordaremos los endpoints, los DTO, los formatos JSON, los códigos HTTP y el tratamiento de errores para evitar incompatibilidades entre frontend y backend.
6. **Integración Continua:** Nos comunicaremos diariamente para conectar el frontend con los controladores REST a medida que se terminen los módulos y realizar pruebas conjuntas.
7. **Control de Versiones:** Trabajaremos mediante ramas de Git, commits descriptivos y pull requests cuando corresponda. Las migraciones deberán versionarse y coordinarse para evitar conflictos.
8. **Pruebas y Validación:** Cada módulo deberá probarse antes de integrarlo con los demás. Las operaciones críticas de ventas, caja e inventario tendrán pruebas específicas de integridad y concurrencia.

5. Hoja de Ruta (Roadmap)

El proyecto se dividirá en cuatro fases (sprints), enfocadas en tener el MVP listo para la primera prueba piloto en un comercio real.

Fase 1: Cimientos y Arquitectura (Semana 1)

- Definición y aprobación del diagrama de base de datos.
- Configuración del repositorio en GitHub.

- Configuración del backend con Java, Spring Boot y Maven.
- Configuración del frontend con React, JavaScript, Vite y Electron.
- Configuración de la conexión con MySQL/MariaDB.
- Configuración de Flyway para las migraciones.
- Creación de las entidades JPA, repositorios y relaciones.
- Creación de datos de prueba para usuarios, categorías y productos.
- Definición de los primeros contratos de la API REST.
- Configuración inicial de seguridad y manejo de errores.

Fase 2: Módulos de Inventario y Caja (Semanas 2-3)

- ABM (CRUD) de productos, categorías y proveedores.
- Gestión de códigos de barras, SKU, precios, costos y stock mínimo.
- Registro de movimientos de inventario.
- Gestión de compras y reposición de mercadería, si se incluye en el alcance inicial.
- Lógica de apertura, movimientos y cierre de caja.
- Registro de ingresos, egresos y medios de pago.
- Desarrollo de las interfaces de gestión interna en React.
- Integración de las interfaces con la API REST.
- Implementación de permisos diferenciados para administradores y cajeros.

Fase 3: El Punto de Venta (Semana 4)

- Desarrollo de la interfaz del mostrador mediante React.
- Implementación del carrito de compras y buscador predictivo.
- Búsqueda de productos por nombre, SKU y código de barras.
- Optimización de la interfaz para el uso con teclado y lector de códigos de barras.
- Implementación de la lógica transaccional crítica mediante @Transactional.
- Control de stock concurrente mediante bloqueos de filas y validación de existencias.

- Registro consistente de ventas, detalles, pagos y movimientos de inventario.
- Cálculo de precios, descuentos y totales exclusivamente en el backend.
- Generación e impresión de comprobantes internos mediante Electron.
- Recuperación del historial y detalle de ventas registradas.

Fase 4: Pruebas de Estrés y Piloto Comercial (Semana 5)

- Pruebas de calidad (QA) sobre las operaciones principales.
- Pruebas de ventas concurrentes y prevención de stock negativo.
- Pruebas de fallos de comunicación entre el frontend y el backend.
- Verificación de la atomicidad de las transacciones ante errores.
- Pruebas de apertura, arqueo y cierre de caja.
- Verificación de los permisos y controles de seguridad.
- Pruebas de impresión de tickets.
- Configuración del empaquetado y del instalador de Electron.
- Definición del procedimiento de instalación del backend y la base de datos.
- Configuración de copias de seguridad y prueba de restauración.
- Instalación del sistema en el "Cliente Cero" para una demostración real.
- Recopilación de observaciones y corrección de problemas detectados durante el piloto.

6. Consideraciones Técnicas Iniciales

- El backend será la única capa responsable de ejecutar las reglas de negocio y acceder directamente a la base de datos.
- El frontend se comunicará con Spring Boot mediante la API REST; nunca accederá directamente a MySQL/MariaDB.
- Los importes monetarios se manejarán con BigDecimal en Java y tipos DECIMAL en la base de datos.
- Las ventas confirmadas no se eliminarán físicamente. Las anulaciones deberán conservar el historial y registrar los movimientos compensatorios necesarios.
- Las operaciones de venta e inventario deberán ejecutarse en una única transacción cuando corresponda.

- Los productos, ventas y movimientos deberán mantener relaciones e integridad referencial.
- Electron se configurará con medidas de seguridad adecuadas, incluyendo aislamiento de contexto y acceso limitado a las funciones del sistema operativo.
- Las credenciales y secretos no se incluirán en el código del frontend ni en el repositorio.
- Se definirá una estrategia de copias de seguridad y recuperación de la base de datos.
- Para el piloto inicial se priorizará una instalación local por comercio, dejando la posibilidad de incorporar una modalidad centralizada en la nube.
- Los tickets del MVP serán comprobantes internos no fiscales. La integración con ARCA quedará fuera del alcance inicial, salvo que se acuerde expresamente.

7. Resultado Esperado

Al finalizar las cinco semanas se espera contar con una aplicación de escritorio instalable, con frontend desarrollado en Electron y React, backend en Java con Spring Boot y persistencia en MySQL/MariaDB.

El sistema deberá permitir administrar productos, controlar inventario, registrar ventas, gestionar caja, imprimir tickets y consultar reportes básicos. Las operaciones críticas deberán mantener la integridad de los datos incluso ante errores y solicitudes concurrentes.

El resultado será una primera versión funcional destinada a validar el producto en un comercio real, identificar mejoras y establecer las bases para su comercialización mediante instalación inicial y abono mensual de mantenimiento y soporte.